Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра „Вычислительная техника“

ОТЧЕТ

по лабораторной работе №6

по курсу „Разработка кроссплатформенных приложений “

на тему „ Сетевое взаимодействие в Java“

Вариант 7

Выполнил студенты группы 23ВВИ1:
Макаров.А.С.
Орищенко М.Д.

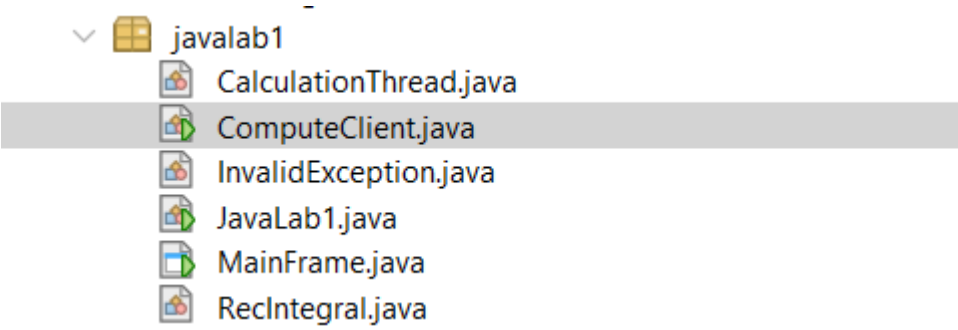
Приняли:
к.т.н. доцент Юрова О.В.

Пенза 2026

Цель работы: научиться создавать клиент-серверные приложения с использованием стандартных классов Java.

Задание: Модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру, обеспечивающую распределенное вычисление определенного интеграла на нескольких вычислительных узлах (клиентах) при этом каждый узел использует несколько нитей, как в предыдущей работе. Сервер не занимается вычислениями, а лишь реализует взаимодействие с пользователем и агрегацию результатов вычислений от клиентов. Нечетные варианты используют протокол UDP, а четные TCP. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:



Фигура 1: Создали новый файл "Клиент"

Код файла:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package javalab1;

import java.net.*;
import java.io.*;
```

```
public class ComputeClient {
    public static void main(String[] args) {
        int port = 12345;
        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Клиент запущен (порт " + port + "). Ожидание
задач...");

            while (true) {
                byte[] buffer = new byte[4096];
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);

                ObjectInputStream ois = new ObjectInputStream(new
ByteArrayInputStream(request.getData()));
                RecIntegral task = (RecIntegral) ois.readObject();

                System.out.println("Получена задача: нижний предел " +
task.getLowerBound());

                new Thread(() -> {
                    task.calculate();
                    try {
                        ByteArrayOutputStream baos = new ByteArrayOutputStream();
                        ObjectOutputStream oos = new ObjectOutputStream(baos);
                        oos.writeObject(task);
                        byte[] responseData = baos.toByteArray();

                        DatagramPacket response = new DatagramPacket(
                            responseData, responseData.length, request.getAddress(),
request.getPort()
```

```
                );
                socket.send(response);
                System.out.println("Результат отправлен серверу.");
            } catch (IOException e) {
                e.printStackTrace();
            }
        }).start();
    }
} catch (Exception e) {
    e.printStackTrace();
}
}

}
```

На стороне клиента: При получении каждого нового UDP-пакета создается новый поток (`new Thread()`), что позволяет клиенту выполнять несколько вычислений параллельно (согласно требованиям ЛР №5).
На стороне сервера: Для приема ответов реализован отдельный фоновый поток `listenForResults`. Это предотвращает "зависание" графического интерфейса пользователя во время ожидания сетевого ответа.

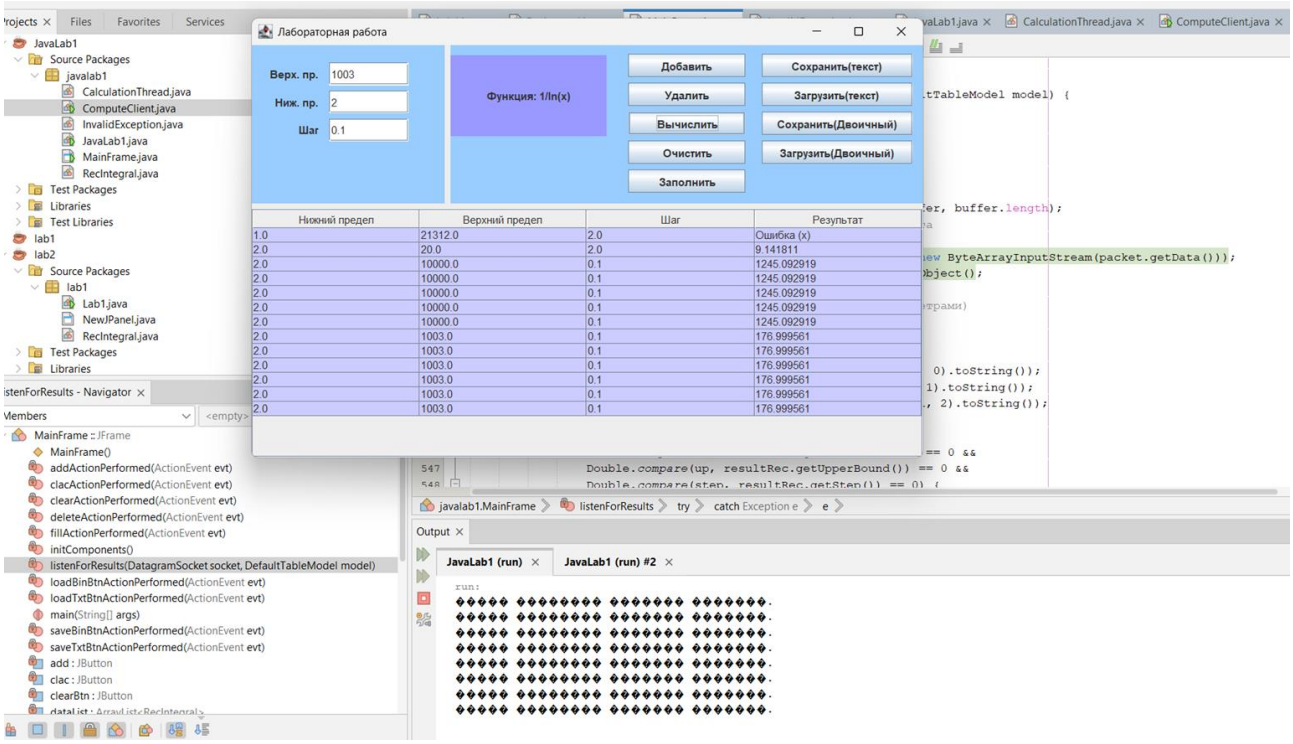
Логика обновления данных
Реализован алгоритм идентификации ответов. Поскольку UDP не гарантирует порядок доставки, сервер ищет нужную строку в `JTable`, сравнивая параметры пришедшего объекта (`lowerBound`, `upperBound`, `step`) с данными в таблице, после чего обновляет ячейку с результатом.

3. Текущее состояние программы

Теперь работа с программой происходит в два этапа:

1. **Запуск вычислительного узла**: Клиент переходит в режим ожидания.
2. **Работа в GUI**: Пользователь добавляет данные в таблицу и нажимает «Вычислить». Сервер рассылает задачи, а затем динамически заполняет таблицу по мере поступления ответов от сетевых узлов.

Программа стала масштабируемой. Теоретически, вычислительных узлов (`ComputeClient`) может быть несколько на разных компьютерах, что позволяет распределять нагрузку и значительно ускорять вычисление большого количества интегралов.



Программа запущена и производит расчеты. Происходит это путем связи хост — клиент.

Вывод: научились создавать клиент-серверные приложения с использованием стандартных классов Java.